EMBEDDED SYSTEM PROJECT

# RTC 시계 및 복합 계산기 시스템 개발

Data Sheet 해석 및 I2C 통신 기반 시스템 통합

김호철, 이규현

# 1. 개요

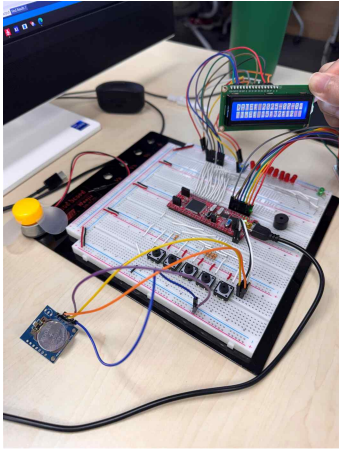
## 개발 목적

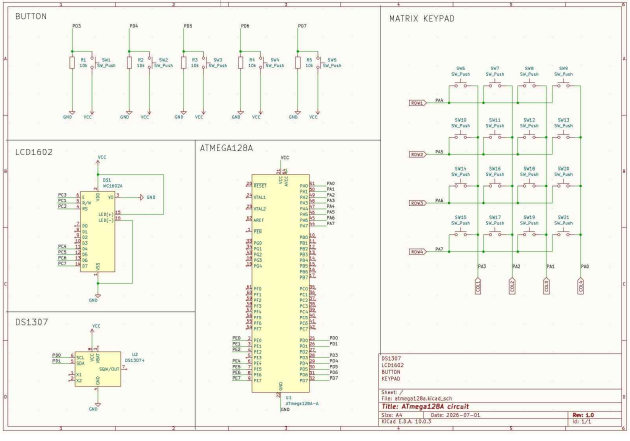
- **데이터시트 해석:** 제조사 제공 문서를 분석하여 레지스터 레벨의 제어 로직을 직접 개발
- **I2C 통신 체득:** DS1307과의 핸드쉐이크 과정을 통해 2-Wire 인터페이스의 물리 계층 동작 원리 이해
- **기능 통합:** 실시간 시간 표출 및 키패드 입력 기반 복합 계산 알고리즘의 시스템 통합

## 개발 환경

- **MCU:** ATmega128 (8-bit AVR)
- **IDE/Tool:** Microchip Studio(Atmel Stduio)
- **Main Comm:** I2C, 4-bit Parallel
- **Analysis:** Oscilloscope (Waveform Debug)

# 2. H/W: 외관 사진 및 시스템 회로도

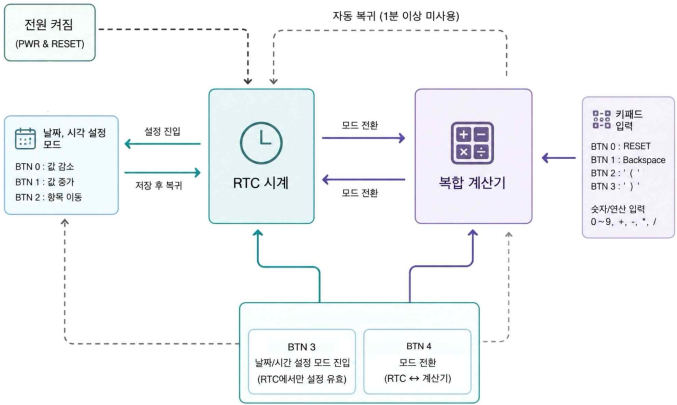




# 2.1 PART LIST

| 부품명 | 상세 사양 | 수량 | 주요 역할 |
|---|---|---|---|
| **LCD1602** | 16x2 Character, 4-bit Parallel | 1 | 데이터 시각화 출력 장치 |
| **DS1307** | I2C Real-Time Clock | 1 | 실시간 시간 데이터 공급 |
| **Push Switch** | Tact Switch (4-Pin) | 5 | 설정 모드 및 계산기 모드 입력 |
| **Resistor** | 1/4W Axial, 10kΩ | 7 | I2C 풀업 및 스위치 풀다운/풀업 저항 |
| **4x4 Keypad** | Matrix Membrane Type | 1 | 계산기 숫자 및 연산 기호 입력 |
| **MCU** | ATmega128A | 1 | 전체 시스템 로직 및 통신 제어 |

# 3.1 S/W: 전체 FSM


전원 켜짐
(PWR & RESET)
자동 복귀 (1분 이상 미사용)
날짜, 시각 설정 모드
BTN 0 : 값 감소
BTN 1 : 값 증가
BTN 2 : 항목 이동
설정 진입
저장 후 복귀
RTC 시계
모드 전환
모드 전환
복합 계산기
키패드 입력
BTN 0 : RESET
BTN 1 : Backspace
BTN 2 : ' ( '
BTN 3 : ' ) '
숫자/연산 입력
0 ~ 9, +, -, *, /
BTN 3
날짜/시간 설정 모드 진입
(RTC에서만 설정 유효)
BTN 4
모드 전환
(RTC ↔ 계산기)

# 3.2.1 LCD Driver: Spec Summary

## 기본 사양 (Source: 4)

**Display:** 16 characters x 2 lines

**Input High:** 0.7 Vcc min.

**Input Low:** 0.55V max.

**E-Cycle Time:** 1000ns min.

**E-Pulse Width:** 450ns min.

## 중요 명령어 (Instruction)

- **0x01:** Clear Display (1.64ms 대기)
- **0x02:** Cursor Return (1.64ms 대기)
- **0x06:** Entry Mode (Cursor Auto Inc)
- **0x28:** Function Set (4-bit, 2-line)
- **0x0C:** Display ON (Cursor OFF)

# 3.2.1 LCD Driver: 제어 및 구조적 사양

### 레지스터 선택 (RS 핀)

**RS = 0: Instruction Reg**

화면 초기화(0x01), 입력 설정 등 명령용 레지스터 지정

**RS = 1: Data Register**

화면에 문자를 직접 표출하기 위한 아스키 코드 저장

### 내부 메모리 아키텍처

**DDRAM (Display RAM)**

문자 코드 저장 영역. 1행은 0x00, 2행은 0x40 주소 할당

**CGROM & CGRAM**

기본 폰트용 CGROM 및 사용자 커스텀 폰트 구현용 CGRAM

### 동기 제어 타이밍 (E 핀)

**Enable (E) 트리거**

데이터를 버스에 로드한 후 E 핀을 Pulse 형태로 제어

**Falling Edge Latch**

E 핀이 High에서 Low로 천이되는 시점에 내부 래치 회로가 캡처

# 3.2.1 LCD 초기화 시퀀스 상세 (4-bit)

데이터시트 명세에 따른 **Software Reset** 및 인터페이스 설정 과정

**STEP 1**
**Power On**
Wait 40ms
(VCC 안정화)

**STEP 2**
**8-bit Reset**
0x30 전송(3회)
(4.1ms / 100us 대기)

**STEP 3**
**4-bit Conv**
0x20 전송
(4비트 모드 진입)

**STEP 4**
**Function Set 4-bit**
0x28 전송
(4-bit, 2-Line, 16-Chars)

**STEP 5**
**Final Set**
Disp ON / Clear / Entry Mode

**※ 상세 분석:** 부팅 초기 LCD는 8비트 모드인지 4비트 모드인지 알 수 없는 상태일 수 있습니다. 따라서 **0x30을 세 번 전송**하는 "Software Reset" 과정을 통해 어떤 명령 상태에 있더라도 강제로 8비트 모드로 동기화시킨 후 4비트로 전환하는 것이 신뢰성 확보의 핵심입니다.

# 3.2.1 Code 구현

```
static void lcd_pulse(void)
{
    // Enable 핀 하이/로우 트리거로 LCD에 데이터 쓰기 완료
    LCD_DATA_PORT |= (1 << LCD_E);
    _delay_us(1);
    LCD_DATA_PORT &= ~(1 << LCD_E);
    _delay_us(1);
}

// 4비트 데이터를 포트에 싣고 가동 펄스(E)를 발생시키는 내부 함수
static void lcd_send_nibble(uint8_t nibble)
{
    // 상위 4비트 영역만 데이터 교체
    LCD_DATA_PORT = (LCD_DATA_PORT & 0x0F) | (nibble & 0xF0);

    lcd_pulse();
}
```

```
// 8비트 데이터를 상위/하위 4비트씩 2회 분할 전송
// (mode: 0=명령어, 1=일반 데이터)
static void lcd_send_byte(uint8_t value, uint8_t mode)
{
    // RS = 0 : 명령어 모드 지정
    // RS = 1 : 데이터 표시 모드 지정
    if (mode == 0) LCD_DATA_PORT &= ~(1 << LCD_RS);
    else           LCD_DATA_PORT |= (1 << LCD_RS);

    // R/W = 0 : 쓰기(Write) 모드로 고정
    LCD_DATA_PORT &= ~(1 << LCD_RW);

    // 1단계: 상위 4비트 먼저 전송
    lcd_send_nibble(value & 0xF0);
    // 2단계: 하위 4비트를 끌어올려 전송
    lcd_send_nibble((value << 4) & 0xF0);

    _delay_us(50);
}
```

# 3.2.1 Code 구현

```
// 데이터시트 규격에 맞춘 4비트 환경 전용 초기화 시퀀스
void lcd_init(void)
{
    // 1. 데이터 및 제어 핀 포트 방향 출력 설정
    LCD_DATA_DDR |= (1 << LCD_RS) | (1 << LCD_RW) | (1 << LCD_E) | 0xF0;
    LCD_DATA_PORT &= ~((1 << LCD_RS) | (1 << LCD_RW) | (1 << LCD_E) | 0xF0);

    _delay_ms(50); // Power On 이후 초기 전압 상승 대기

    // 2. 소프트웨어 리셋: 8비트 모드 명령(0x30)을 강제로 3회 주입해 내부 칩 리셋
    lcd_send_nibble(0x30);
    _delay_ms(5);
    lcd_send_nibble(0x30);
    _delay_us(150);
    lcd_send_nibble(0x30);
    _delay_us(150);

    // 3. 4비트 모드 가동 명령(0x20) 전송
    lcd_send_nibble(0x20);
    _delay_ms(2);

    // 4. 레지스터 환경 설정 명령 순차 송신
    lcd_command(LCD_CMD_FUNCTION_SET);  // 4비트 버스, 2줄 출력 설정
    lcd_command(LCD_CMD_DISPLAY_ON);    // 디스플레이 화면 활성화
    lcd_clear();                        // 화면 청소
    lcd_command(LCD_CMD_ENTRY_MODE);    // 주소 자동 증가 방향 설정
}
```

# 3.2.1 Code 구현

```
// 행/열 좌표로 커서를 이동시키는 함수 (row: 0=첫째줄, 1=둘째줄 / col: 0~15)
void lcd_goto_xy(uint8_t row, uint8_t col)
{
    uint8_t address = 0;

    if (row == 0) address = 0x00 + col; // 1번째 줄 시작 주소 기반 연산
    else          address = 0x40 + col; // 2번째 줄 시작 주소 기반 연산

    lcd_command(0x80 | address); // DDRAM 주소 지정 커맨드 비트(0x80)와 결합하여 전송
}

// 하드웨어 커서 모드 제어 함수 (visible: 밑줄 생성, blink: 네모 박스 깜빡임)
void lcd_set_cursor_mode(uint8_t visible, uint8_t blink)
{
    uint8_t command = LCD_CMD_DISPLAY_ON; // 기본 디스플레이 On 상태 베이스

    if (visible) command |= 0x02; // 커서 밑줄 활성화
    if (blink)   command |= 0x01; // 커서 블링크 네모 박스 활성화

    lcd_command(command);
}
```

# 3.2.2 DS1307(I2C) RTC: Spec Summary

## 주요 기능 (Source: 3)

**Interface:** 2-Wire Serial (I2C)

**Addr:** 0x68 (1101000)

**BCD Format:** 모든 시간 데이터는 BCD 코드로 관리

**Battery Backup:** 전원 차단 시 배터리 자동 전환

## Register Map

| ADDR | BIT 7 | Function |
|---|---|---|
| 00h | CH | Seconds (CH: Halt) |
| 01h | X | Minutes (00-59) |
| 02h | 12/24 | Hours (12/24 Mode) |
| 07h | OUT | Control (SQW/OUT) |

⚠ 주의: 00h 레지스터의 CH(Clock Halt) 비트가 1이면 시간이 흐르지 않으므로 초기화 시 반드시 0으로 클리어해야 함.

# 3.2.2 RTC I2C 통신 동작 시나리오

I2C 하위 레벨 제어를 통한 데이터 전송 흐름 분석

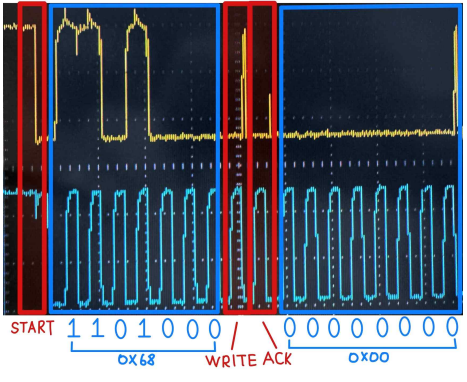
## [ Write ]

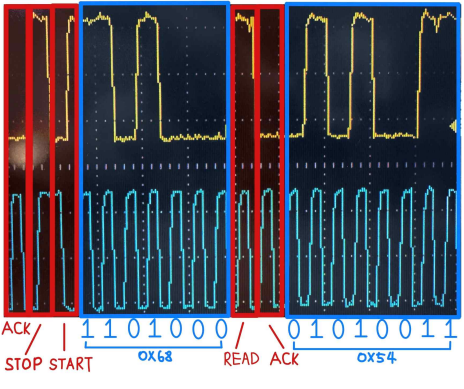



## [ Read ]



# 3.2.2 오실로스코프 파형 분석

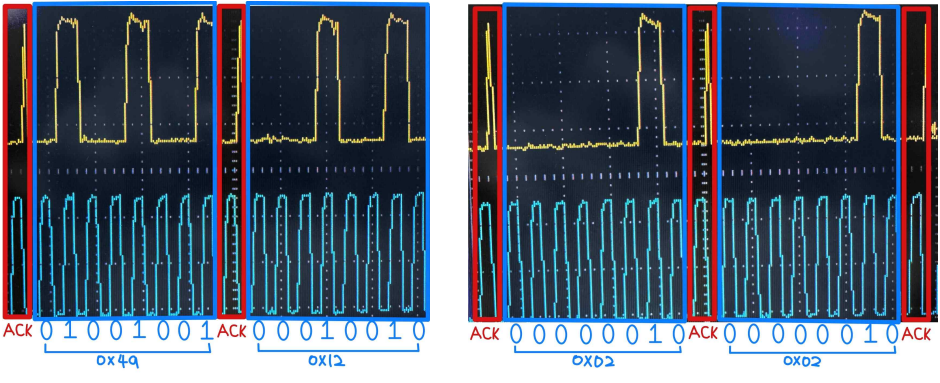



**시작 주소 지정** : START -> 0x68(addr) + WRITE -> ACK -> 0x00 -> ACK -> STOP

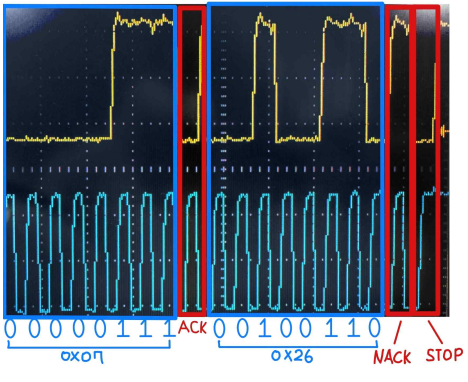
**READ 시작** : START -> 0x68(addr) + READ -> ACK -> 0x54(second)

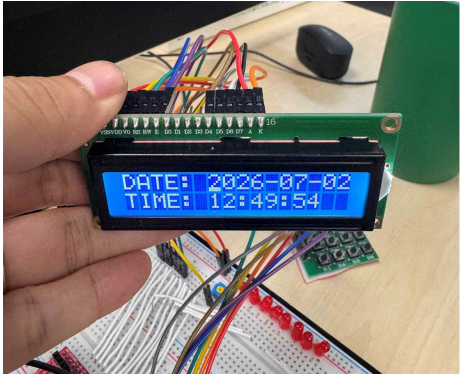
# 3.2.2 오실로스코프 파형 분석




ACK -> 0x49(minutes) -> ACK -> 0x12(hours) -> ACK -> 0x02(day) -> ACK -> 0x02(date) -> ACK

# 3.2.2 오실로스코프 파형 분석




0x07(month) -> ACK -> 0x26(year) -> NACK -> STOP

# 3.2.2 Code 구현

```
// RTC 칩 초기화 및 배터리 백업 판단 함수
void ds1307_init(date_time_t *initial_time)
{
    I2C_init(); // I2C 하드웨어 모듈 가동

    // 1. 현재 내부 포인터를 00h(초) 레지스터 위치로 이동
    I2C_start();
    I2C_transmit(SLAVE_WRITE_ADDR);     // 쓰기 주소 전송
    I2C_transmit(0);                    // 0번 주소방 지정
    I2C_stop();

    // 2. 00h번지(초) 레지스터 값 1바이트 읽기
    I2C_start();
    I2C_transmit(SLAVE_READ_ADDR); // 읽기 주소 전송
    uint8_t second = I2C_receive_NACK();
    I2C_stop();

    // 3. 7번 비트(CH) 검사를 통한 시계 상태 판별
    if (second & 0x80)
    {
        // CH 비트가 1이면 시계가 멈춘 상태 -> 초기 시간 세팅 및 오실레이터 가동
        ds1307_set_time(initial_time);
    }
    else
    {
        // CH 비트가 0이면 정상 구동 중 상태 -> 배터리로 가고 있던 진짜 시간을 변수에 역복사
        ds1307_get_time(initial_time);
    }
}
```

# 3.2.2 Code 구현

```c
// 하드웨어 RTC에서 실시간으로 시각 데이터를 읽어오는 함수
void ds1307_get_time(date_time_t *time_data)
{
    // 1. 읽어올 시작 메모리 주소를 0번지(초)로 강제 지정
    I2C_start();
    I2C_transmit(SLAVE_WRITE_ADDR);
    I2C_transmit(0);
    I2C_stop();

    // 2. 읽기 모드로 다시 접근하여 데이터 순차 수신
    I2C_start();
    I2C_transmit(SLAVE_READ_ADDR);

    // 칩 내부 주소가 자동 증가하므로 연속 수신 가능 (ACK로 응답)
    uint8_t second  = I2C_receive_ACK();        // 00h: 초
    uint8_t minute  = I2C_receive_ACK();        // 01h: 분
    uint8_t hour = I2C_receive_ACK();           // 02h: 시
    uint8_t day_of_week = I2C_receive_ACK();    // 03h: 요일 (스킵)
    uint8_t day   = I2C_receive_ACK();          // 04h: 일
    uint8_t month  = I2C_receive_ACK();         // 05h: 월
    uint8_t year = I2C_receive_NACK();          // 06h: 년 (마지막이므로 NACK 전송)
    I2C_stop();

    // 3. 수신된 BCD 데이터를 10진수로 변환하여 메인 구조체에 탑재
    time_data->second       = bcd_to_decimal(second & 0x7F);    // CH 비트 제외 필터링
    time_data->minute       = bcd_to_decimal(minute);
    time_data->hour         = bcd_to_decimal(hour & 0x3F);      // 24시간 모드 비트 필터링
    time_data->day_of_week  = bcd_to_decimal(day_of_week);
    time_data->day          = bcd_to_decimal(day);
    time_data->month        = bcd_to_decimal(month);
    time_data->year         = 2000 + bcd_to_decimal(year);      // 2자리 연도를 4자리 연도로 가공
}
```

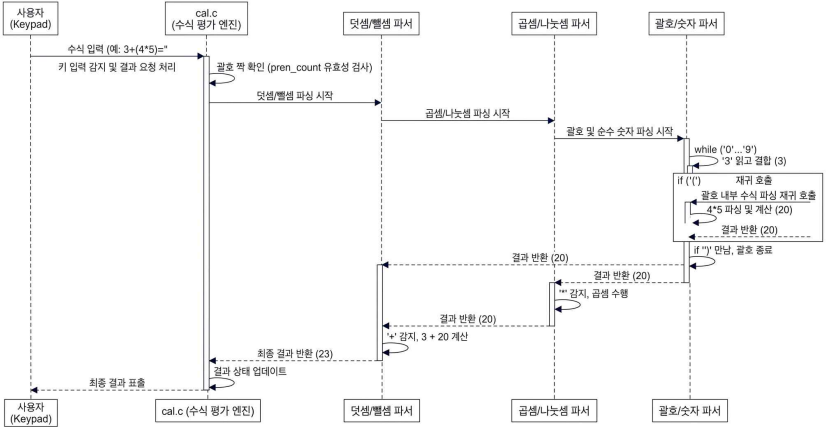
# 3.2.2 Code 구현

```
// 사용자가 설정한 변경 시간을 하드웨어 RTC에 덮어쓰는 함수
void ds1307_set_time(const date_time_t *time_data)
{
    // 1. 기록할 시작 메모리 주소를 0번지(초)로 지정
    I2C_start();
    I2C_transmit(SLAVE_WRITE_ADDR);
    I2C_transmit(0);

    // 2. 10진수 시간 데이터를 BCD로 포맷팅하여 순서대로 전송
    I2C_transmit(decimal_to_bcd(time_data->second) & 0x7F); // 7번 비트를 0으로 밀어 시계 가동(CH=0)
    I2C_transmit(decimal_to_bcd(time_data->minute));
    I2C_transmit(decimal_to_bcd(time_data->hour) & 0x3F);   // 24시간 모드로 강제 지정
    I2C_transmit(decimal_to_bcd(time_data->day_of_week));
    I2C_transmit(decimal_to_bcd(time_data->day));
    I2C_transmit(decimal_to_bcd(time_data->month));
    I2C_transmit(decimal_to_bcd(time_data->year % 100));    // 4자리 연도를 하위 2자리 숫자로 압축 변환

    I2C_stop(); // 정지 신호 전송
}
```

# 3.2.3 계산기: Sequence Chart



# 3.2.3 계산기 알고리즘: Stack & Queue

## 후위 표기법 (Postfix)

복합 수식의 우선순위 해결을 위해 중위 표기법(Infix)을 후위 표기법으로 변환

**Infix:** 3 + 4 * 2

**Postfix:** 3 4 2 * +

## 구현 메커니즘

**STACK:** 연산자(+,-,*,/) 및 괄호의 우선순위 관리

**QUEUE:** 변환된 수식의 토큰 순서대로 저장

**Algorithm:** 피연산자는 큐로, 연산자는 스택 우선순위 비교 후 팝/푸시

# 3.2.2 Code 구현

```
// 키패드 문자 입력 처리 (숫자, 사칙연산자, 등호 분기)
void cal_input_key(uint8_t key)
{
    // 결과가 나온 상태에서 새 숫자를 누르면 화면 자동 초기화
    if (should_reset)
    {
        if ((key >= '0' && key <= '9') || key == '+' || key == '-' || key == '*' || key == '/')
        {
            cal_reset();
        }
    }

    // 숫자 또는 사칙연산 부호 기호 배열 적재
    if ((key >= '0' && key <= '9') || key == '+' || key == '-' || key == '*' || key == '/')
    {
        if (expr_idx < (int16_t)sizeof(expr) - 1)
        {
            expr[expr_idx++] = key;
            expr[expr_idx] = '\0';
        }
    }
    // '=' 입력 시 파서 작동 및 결과 산출 시동
    else if (key == '=')
    {
        expr[expr_idx] = '\0';
        evaluated_result = evaluate_calc();
        is_evaluated = 1;
        should_reset = 1;
    }
}
```

# 3.2.2 Code 구현

```
// 괄호 유효성 선행 검사 후 연산 시동
static int64_t evaluate_calc(void)
{
    int16_t paren_count = 0;

    // 연산 출발 전 괄호 개수 짝이 완벽한지 수식 전체 스캔
    for (int16_t i = 0; i < expr_idx; i++)
    {
        if (expr[i] == '(')       paren_count++;
        else if (expr[i] == ')')  paren_count--;

        if (paren_count < 0) break;
    }

    // 괄호 열고 안 닫았으면 연산 중단하고 즉시 에러 반환
    if (paren_count != 0)
    {
        has_error = 1;
        return 0;
    }

    parse_idx = 0;
    has_error = 0;

    return parse_add_sub(); // 최하위 레벨(덧셈/뺄셈) 시작
}
```

# 3.2.2 Code 구현

```
// 3단계: 최하위 우선순위 연산 (덧셈, 뺄셈 처리)
static int64_t parse_add_sub(void)
{
    int64_t value = parse_mul_div();

    while (expr[parse_idx] == '+' || expr[parse_idx] == '-')
    {
        char op = expr[parse_idx++];
        int64_t rhs = parse_mul_div();

        if (op == '+')
        {
            value = value + rhs;
        }
        else
        {
            value = value - rhs;
        }
    }

    return value;
}
```

# 3.2.2 Code 구현

```
// 2단계: 중간 우선순위 연산 (곱셈, 나눗셈 처리)
static int64_t parse_mul_div(void)
{
    int64_t value = parse_bracket();

    while (expr[parse_idx] == '*' || expr[parse_idx] == '/')
    {
        char op = expr[parse_idx++];
        int64_t rhs = parse_bracket();

        if (op == '*')
        {
            value = value * rhs;
        }
        else
        {
            value = (rhs != 0) ? value / rhs : 0; // 0으로 나누기 뻗음 예방
        }
    }

    return value;
}
```

# 3.2.2 Code 구현

```
// 파싱 1단계: 최우선순위 연산 (순수 숫자 결합 및 괄호 처리)
static int64_t parse_bracket(void)
{
    int64_t value;

    // 괄호를 만나면 parse_add_sub을 재귀 호출하여 괄호 속 수식 먼저 격파
    if (expr[parse_idx] == '(')
    {
        parse_idx++;
        value = parse_add_sub();

        if (expr[parse_idx] == ')') parse_idx++;
        else has_error = 1; // 짝 괄호 탈출 실패 시 에러
    }
    else
    {
        value = 0;

        // 문자 기호로 들어온 연속된 숫자를 하나의 10진수 값으로 결합
        while (expr[parse_idx] >= '0' && expr[parse_idx] <= '9')
        {
            int64_t digit = expr[parse_idx] - '0';

            value = value * 10 + digit;
            parse_idx++;
        }
    }

    return value;
}
```

# 3.2.2 Code 구현

```
static void int_to_str(int64_t num, char *str)
{
    char temp[25];
    int16_t i = 0, j = 0;
    uint8_t is_negative = 0;

    // 입력값이 0인 경우 예외 처리 후 즉시 종료
    if (num == 0)
    {
        str[0] = '0';
        str[1] = '\0';
        return;
    }

    // 음수 처리: 마이너스 부호 플래그를 켜고 양수로 반전
    if (num < 0)
    {
        is_negative = 1;
        num = -num;
    }

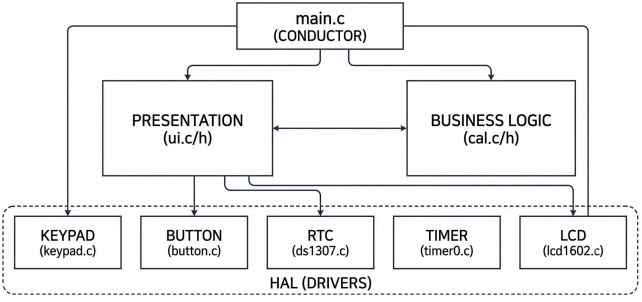
    // 숫자를 10으로 나누며 뒷자리부터 한 글자씩 temp 배열에 거꾸로 적재
    while (num > 0)
    {
        temp[i++] = (num % 10) + '0'; // 숫자를 아스키(ASCII) 문자로 변환
        num /= 10;
    }

    // 음수였던 숫자는 최종 문자열(str) 맨 앞에 마이너스 부호 먼저 주입
    if (is_negative)
    {
        str[j++] = '-';
    }

    // temp 배열에 거꾸로 뒤집혀 담긴 문자들을 순서대로 꺼내어 str에 정방향 복사
    while (i > 0)
    {
        str[j++] = temp[--i];
    }

    str[j] = '\0'; // NULL(\0) 문자 마감 처리
}
```

# 4. 전체 구조




```c
int main(void)
{
    system_init();

    sei();

    last_input_ms = ms_count;

    while (1)
    {
        // 버튼 처리
        task_process_buttons();

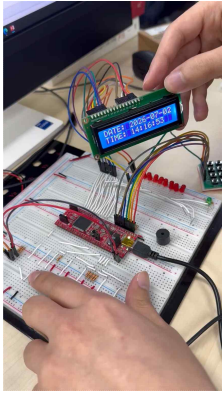
        // 시간 동기화
        task_sync_rtc();

        // 키패드 입력 처리
        task_process_keypad();

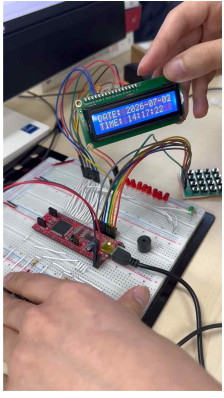
        // 계산기 입력 타임아웃 체크
        task_check_timeout();

        // 디스플레이 표출
        task_render_display();
    }
}
```

# 5. 동작 영상

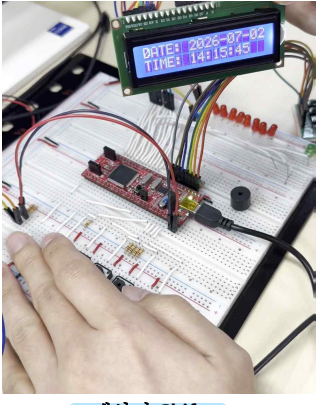



Reset 동작

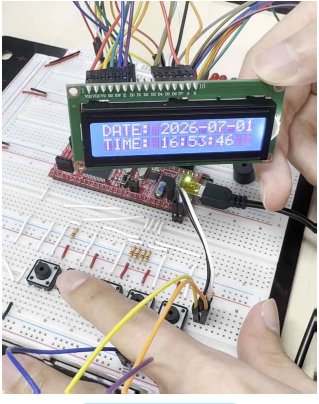



계산기 대기 동작

# 5. 동작 영상




계산기 Shift 동작




전체 동작

# 6. 구현 후기

**[LCD 하드웨어 시프트]** 명령어의 물리적 실행 시간(1.64ms) 제약을 고려하여 4비트 인터페이스의 타이밍 동기화를 정밀하게 제어함으로써, 긴 수식 입력 시 화면이 부드럽게 밀리는 하드웨어 스크롤 기능을 성공적으로 구현했습니다.

**[오버플로우 방지]** 정수 범위 한계를 극복하기 위해 문자열 변환 버퍼 함수로 제어하여 큰 자릿수도 깨짐없이 정상 계산되도록 완성했습니다.

**[데이터시트 및 I2C 후기]** 데이터시트의 타이밍 스펙을 엄격히 준수하는 것이 시스템 신뢰성의 핵심임을 깨달았고, 오실로스코프로 I2C 버스의 실시간 BCD 파형을 직접 계측 및 분석해 보며 임베디드 통신 제어의 깊이를 더한 값진 경험이었습니다.

# 감사합니다